

Scaling Techniques Documentation

1. Overview

This document explains the scaling technique applied to the ATS customer churn dataset during data preparation, so the Data Analyst can reproduce or extend the pipeline without refitting the scaler from scratch. It accompanies the preprocessed dataset and the train/test split files in this submission.

2. Columns Scaled

Only continuous numeric columns were scaled. Binary, ordinal, and one-hot encoded columns were left untouched. Scaling them would distort their meaning (e.g. turning a 0/1 flag into an arbitrary decimal).

Column	Type	Scaled?
tenure	Continuous (months)	Yes
MonthlyCharges	Continuous (\$)	Yes
TotalCharges	Continuous (\$), engineered = tenure × MonthlyCharges	Yes
gender, SeniorCitizen, Dependents, PhoneService, MultipleLines, Churn	Binary (0/1)	No
Contract	Ordinal (0/1/2)	No
internet_fiber, TenureGroup_13-24m, TenureGroup_25-48m, TenureGroup_49-72m, HighValueCustomer	One-hot / flag (0/1)	No

3. Technique Chosen: StandardScaler

StandardScaler (z-score standardization) was used: each value is transformed to $(x - \text{mean}) / \text{std}$, so the scaled column has mean 0 and standard deviation 1.

Rationale: skew of the three scaled columns, computed during EDA:

Column	Skew	Interpretation
tenure	0.24	Close to 0, fairly symmetric
MonthlyCharges	-0.22	Close to 0, fairly symmetric
TotalCharges	0.96	Moderately right-skewed (product of the other two), but not extreme

None of the three showed heavy skew or extreme outliers, so StandardScaler is appropriate and was kept consistent across all three rather than mixing scalers. StandardScaler is also the conventional default for the linear/logistic-regression-style models typically used on churn data. MinMaxScaler would be the alternative if a downstream model specifically required a bounded [0, 1] range (e.g. distance-based models like KNN).

4. Preventing Data Leakage

The scaler is fit only on the training set (fit_transform), then applied to the test set using the same fitted parameters (transform, never fit_transform). Fitting on test data would leak its distribution into the "learned" mean/std, giving an overly optimistic evaluation later.

5. Code Snippets

Feature engineering (creates TotalCharges, prior to scaling):

```
df['TotalCharges'] = df['tenure'] * df['MonthlyCharges']
```

Train/test split (before scaling, to avoid leakage):

```
from sklearn.model_selection import train_test_split

X = df.drop(columns=['Churn'])
y = df['Churn']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42
)
```

Scaling (fit on train only, transform both):

```
from sklearn.preprocessing import StandardScaler

cols_to_scale = ['tenure', 'MonthlyCharges', 'TotalCharges']
scaler = StandardScaler()

X_train_scaled = X_train.copy()
X_test_scaled = X_test.copy()

X_train_scaled[cols_to_scale] = scaler.fit_transform(X_train[cols_to_scale])
X_test_scaled[cols_to_scale] = scaler.transform(X_test[cols_to_scale])
```

6. Fitted Scaler Parameters

Computed from the training split (test_size=0.2, stratify=y, random_state=42). The Data Analyst should reuse these exact values (or the saved scaler object) rather than refitting, so any new data stays on the same scale as this train/test set.

Column	Mean (train)	Std (train)
tenure	32.36	24.56
MonthlyCharges	64.54	30.26
TotalCharges	2269.87	2263.51

7. Train/Test Split Summary

	Rows	Churn rate
Train	5,634 (80%)	26.54%
Test	1,409 (20%)	26.54%

Stratifying on Churn keeps the same ~26.5% churn rate in both sets, matching the full dataset's base rate.

8. Related Engineered Features (context)

Two other features were engineered alongside TotalCharges; they are not scaled (see Section 2) but are documented here for completeness:

- **TenureGroup:** Tenure binned into 0-12m / 13-24m / 25-48m / 49-72m, one-hot encoded (0-12m dropped as baseline). Represents customer lifecycle stage.

- HighValueCustomer: flag for MonthlyCharges > 80. Represents a revenue-priority segment; EDA showed this group churns at 33.7% vs 22.4% for others.

9. Class Imbalance

Churn is imbalanced: ~26.5% Yes vs ~73.5% No. This was deliberately not corrected during data preparation as scaling and class balancing are separate concerns, and rebalancing is a modeling decision that depends on the model and metric the analyst chooses, not a data-prep step.

Points for the analyst to consider when modeling:

- Accuracy alone will be misleading on this imbalance (a model predicting "No" for everyone scores ~73.5%). Prefer precision/recall, F1, or ROC-AUC.
- If resampling is desired (e.g. SMOTE, random oversampling/undersampling), apply it to the training set only, after the train/test split already provided never to the test set, and never before splitting.
- Alternatively, many classifiers support class_weight='balanced' (e.g. scikit-learn's LogisticRegression, RandomForestClassifier) as a resampling-free way to account for the imbalance.
- The provided train/test split is stratified on Churn, so both sets already preserve the same ~26.5% rate and any resampling should be layered on top of this split, not replace it.